

UNIVERSIDAD DE LAS AMÉRICAS

Ingeniería en Software

Programación III

Análisis y diseño del proyecto Dizeo

Sistema de planificación y asignación de trabajadores y actividades de obra

Análisis y selección de la solución
Empresa del caso Dizeo

Autores: Daniel Valarezo, Esteban Figueroa, Julián Jiménez

Fecha: 18 de octubre de 2026

Quito Ecuador

1. FORMULACIÓN DEL PROBLEMA

Dizeo es el caso de estudio de una empresa de construcción y acabados que atiende Quito y sus valles. El documento de requerimientos describe servicios de pintura, impermeabilización, gypsum, obra gris, señalización y cerramientos. Cada obra reúne actividades que necesitan personas con especialidades determinadas y fechas compatibles con los compromisos del cliente. Cuando varios frentes comparten trabajadores, una decisión local puede afectar la ejecución de otra obra. El problema se concentra en coordinar esa capacidad limitada, no en registrar únicamente nombres de clientes o trabajos (Valarezo et al., 2026).

La necesidad identificada consiste en contar con información consistente para decidir quién puede realizar una actividad, durante qué intervalo y después de cuáles trabajos previos. El caso plantea posibles dobles asignaciones, actividades sin responsable y cambios que no llegan oportunamente a los involucrados. Estas situaciones constituyen hipótesis de diagnóstico: todavía no existen entrevistas o mediciones que permitan afirmar su frecuencia en Dizeo. Durdyev y Hosseini (2020) identifican la comunicación deficiente, la falta de coordinación y la planificación inadecuada entre causas recurrentes de retrasos en construcción. Su revisión justifica investigar esas variables, pero no prueba que todas ocurran en esta empresa.

Los actores principales son la dirección, el supervisor y los trabajadores. La dirección necesita conocer los compromisos y la capacidad disponible. El supervisor

organiza el plan semanal, valida especialidades y resuelve cambios diarios. Los trabajadores informan disponibilidad, ejecutan tareas y reportan avances o bloqueos. Los clientes son afectados por las fechas y reprogramaciones, aunque no se prevé que operen la primera versión. Una asignación incompatible puede producir desplazamientos innecesarios, esperas o redistribución de carga. La magnitud económica de esos efectos deberá medirse antes de calcular ahorros atribuibles al proyecto.

Entre las causas por contrastar están la dispersión de información sobre horarios, la ausencia de una verificación uniforme de especialidades y la falta de actualización de dependencias. Una actividad puede ser urgente y, aun así, no estar lista para empezar. La revisión de Gómez Sánchez et al. (2023) muestra que la programación de varios proyectos con recursos compartidos combina restricciones de recursos y precedencias. Esta relación permite delimitar el caso: una persona no puede atender dos actividades incompatibles, y una tarea sucesora necesita respetar sus trabajos previos. No se presupone que una regla de prioridad produzca por sí sola el mejor calendario global.

Los datos principales son obras, fechas comprometidas, actividades, duración estimada, personal requerido, especialidades, jornadas, ausencias y asignaciones. También se requieren motivos de cambio, tiempos de traslado, avances validados e incidencias. Los procesos relevantes ocurren al preparar el plan semanal, confirmar una asignación, comunicar una reprogramación y comprobar el cierre. Hendrickson (2008) explica que las restricciones de recursos condicionan la programación y que

la revisión interactiva del calendario puede ser necesaria. En este caso, la duración y los traslados se registrarán como estimaciones del supervisor, cuya calidad deberá contrastarse con la ejecución.

Las variables cuantitativas incluyen obras activas, horas disponibles y asignadas por trabajador, conflictos detectados, actividades sin responsable, minutos de planificación y cumplimiento semanal. Las variables cualitativas comprenden especialidad, prioridad, estado, zona y motivo de bloqueo. Se trabajará con intervalos de inicio y fin en la zona America/Guayaquil. Las jornadas disponibles y los traslados restringen qué secuencias resultan viables. La evaluación propuesta contempla observar cuatro semanas antes del piloto y cuatro durante su uso. Una reducción de conflictos deberá compararse con la carga de trabajo de cada período para evitar atribuir mejoras a una menor cantidad de obras.

El proyecto académico conserva Java Swing y PostgreSQL como restricciones tecnológicas del documento base. Los volúmenes de 100 obras, 200 trabajadores, 10 000 actividades y 20 usuarios simultáneos son datos de prueba propuestos, no cifras verificadas de la empresa. La simultaneidad agrega una dificultad: dos supervisores podrían evaluar la misma disponibilidad y confirmar decisiones incompatibles. La documentación de PostgreSQL Global Development Group (s. f.) describe bloqueos de filas que permiten coordinar transacciones concurrentes. Por tanto, validar una agenda visible en pantalla no bastará para garantizar integridad en el momento de confirmar.

La dimensión social se relaciona con claridad de horarios, carga de trabajo y adaptación de los usuarios al registro diario. La dimensión económica aparece en esperas y reprocesos potenciales. Los desplazamientos podrían tener un efecto ambiental, pero no se cuantificará una reducción de emisiones sin distancias y mediciones. Tampoco se atribuyen características culturales no documentadas al personal. El problema queda formulado como la dificultad de coordinar actividades y trabajadores entre obras bajo restricciones de disponibilidad, especialidad, precedencia y traslado, manteniendo información actualizada y responsabilidades verificables.

2. ALCANCE Y MÓDULOS FUNCIONALES

La primera versión conserva los cinco módulos del Word. Incluye búsqueda, filtrado, priorización, detección de ciclos, validación temporal y cálculo de indicadores. Quedan fuera nómina, compras, facturación, rutas óptimas, aplicación móvil y optimización global automática. Las reglas no funcionales originales siguen vigentes; su verificación se planifica para etapas posteriores.

M1 Obras

Propósito: Identificar frentes y compromisos. Actor: Administrador y supervisor.

Datos: Código, nombre, servicio, dirección, contacto, fechas, estado y versión.

Crear obra con código único (RF-01).

Filtrar por estado, servicio, zona y fechas (RF-02).

Modificar fechas con motivo y versión anterior (RF-03).

Dar de baja y exigir resolver asignaciones vigentes (RF-04).

Ordenar por entrega e identificar tareas vencidas o sin responsable (RF-05).

Reglas: Fin no anterior al inicio. Una obra terminada necesita reapertura autorizada. Baja lógica, sin borrar historial.

Relación: M3 aporta actividades; M4 aporta plan y responsables; M5 aporta avance.

Responsable: Daniel Valarezo.

M2 Trabajadores y disponibilidad

Propósito: Mantener capacidad de trabajo verificable. Actor: Administrador y supervisor.

Datos: Código, contacto, especialidades, estado, jornadas, bloqueos y cuenta de usuario.

Registrar trabajador y cuenta con rol (RF-06).

Buscar por especialidad, estado y fecha disponible (RF-07).

Actualizar datos y validar solicitudes de disponibilidad (RF-08).

Desactivar y resolver asignaciones futuras (RF-09).

Calcular horas libres/asignadas y filtrar candidatos compatibles (RF-10).

Reglas: Solo especialidades validadas. Un bloqueo no se aprueba sin resolver cruces. Se rechazan horarios de duración nula o negativa.

Relación: M4 consume la disponibilidad; M5 recoge novedades; seguridad controla los roles.

Responsable: Esteban Figueroa.

M3 Actividades y prioridades

Propósito: Descomponer obras y definir un orden ejecutable. Actor: Supervisor.

Datos: Código, obra, descripción, duración, especialidad, cantidad de personas, fecha, prioridad, estado y dependencias.

Crear tarea de duración positiva en obra activa (RF-11).

Filtrar por obra, estado, prioridad o fecha (RF-12).

Modificar duración, prioridad y dependencias con revisión de asignaciones (RF-13).

Cancelar y liberar reservas futuras (RF-14).

Priorizar por alta–media–baja, fecha y código; detectar ciclos (RF-15).

Reglas: No se aceptan ciclos ni dependencia de una tarea consigo misma. Para esta versión se proponen dependencias dentro de la misma obra. Iniciar exige predecesoras terminadas.

Relación: M1 contiene las tareas; M4 consulta precedencias y prioridad; M5 valida terminaciones.

Responsable: Julián Jiménez.

M4 Planificación y asignación

Propósito: Asignar personal y horarios sin incompatibilidades. Actor: Supervisor.

Datos: Actividad, conjunto de trabajadores, inicio, fin, traslados, estado y versión del plan.

Confirmar validando especialidad, cantidad, disponibilidad y secuencia (RF-16).

Consultar agenda diaria/semanal por persona y obra (RF-17).

Reprogramar con motivo, revalidación y actualización de agendas (RF-18).

Cancelar reserva futura conservando historia (RF-19).

Detectar cruce, identificar reserva conflictiva y sugerir personas libres (RF-20).

Reglas: Solo supervisor confirma. Confirmación de todos los trabajadores en una transacción: todo o nada. Si faltan candidatos, la actividad queda pendiente. Traslados explícitos entre obras.

Relación: Integra M1–M3 y reacciona a bloqueos/avances de M5. Se coordina con M2 para validar la disponibilidad.

Responsable: Esteban Figueroa.

M5 Seguimiento e incidencias

Propósito: Distinguir avance informado de avance validado. Actor: Trabajador y supervisor.

Datos: Reporte, actividad, autor, fecha, porcentaje, evidencia opcional, incidencia, revisión y versiones.

Registrar avance entre 0 y 100 e incidencias (RF-21).

Consultar historial y filtrar incidencias (RF-22).

Corregir con motivo y validar, rechazar o pedir aclaración (RF-23).

Anular sin borrar evidencia (RF-24).

Calcular cumplimiento, señalar vencimientos y validar cierre (RF-25).

Reglas: Trabajador reporta solo tareas asignadas. Cierre requiere 100 % validado y ninguna incidencia abierta. El avance de obra pondera duración vigente y excluye canceladas.

Relación: M3 recibe estado de ejecución; M4 recibe novedades; M1 muestra indicadores.

Responsable: Daniel Valarezo.

3. MODELO ORIENTADO A OBJETOS Y UML PRELIMINAR

El modelo relaciona las entidades del negocio con los cinco módulos. Las clases de dominio contienen sus reglas locales y un servicio de planificación coordinará las validaciones entre objetos. Usuario representa el rol mediante un atributo, sin crear una jerarquía artificial de administradores, supervisores y trabajadores. La composición Obra–Actividad expresa pertenencia exclusiva y conserva la baja lógica.

Se proponen diez clases de dominio: Obra, Actividad, Trabajador, Asignacion, Disponibilidad, Usuario, ReporteAvance, Incidencia, PlanSemanal y EventoAuditoria. Sus atributos y métodos representativos se muestran en las dos vistas siguientes.

3.1. Clases principales y asignación

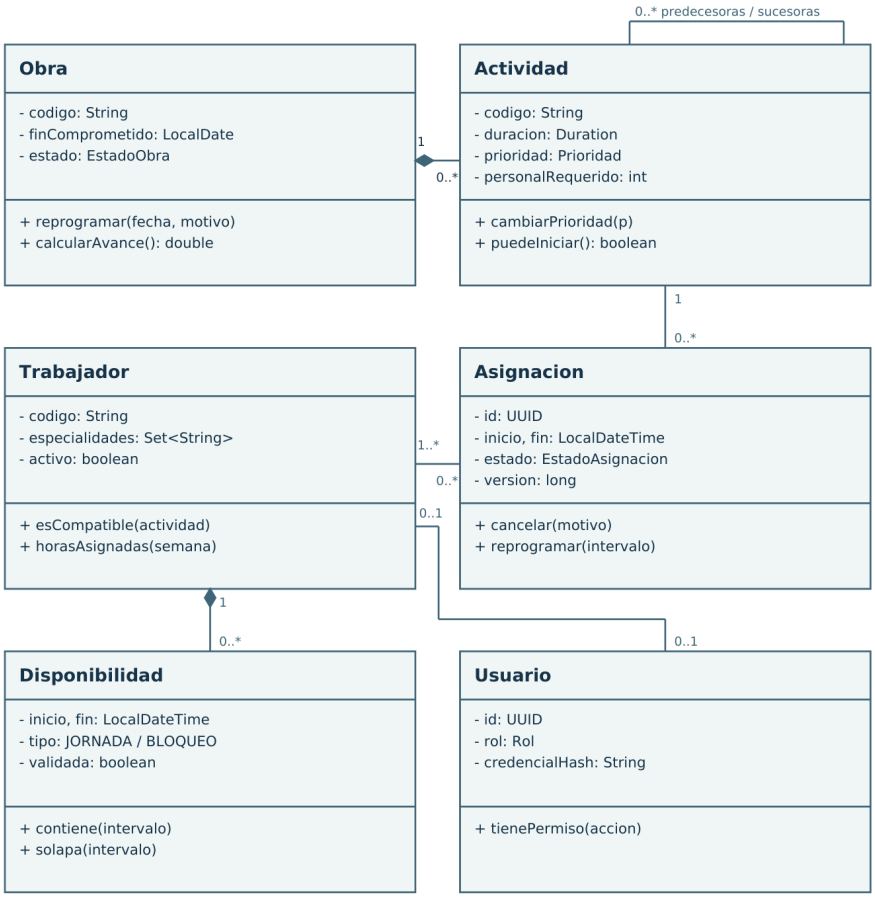


Figura 1. Clases principales, atributos, métodos y relaciones. Fuente: elaboración a partir del documento de requerimientos de Dizeo.

3.2. Seguimiento y versiones del plan



Figura 2. Seguimiento, usuarios y versiones. Actividad y Usuario son las mismas clases de la vista anterior. Fuente: elaboración a partir del documento de requerimientos.

PlanSemanal referencia actividades comprometidas y asignaciones. Las versiones publicadas conservan instantáneas del compromiso y del calendario. ReporteAvance diferencia el porcentaje informado del validado. El cierre de una actividad exige avance validado del 100 % y ninguna incidencia abierta.

3.3. Asociaciones y multiplicidades

Relación	Multiplicidad y significado
Obra–Actividad	1 obra contiene 0..* actividades. Cada actividad pertenece a 1 obra. Composición con baja lógica.
Actividad–Actividad	0..* predecesoras y 0..* sucesoras. Grafo dirigido acíclico dentro de una obra.
Actividad–Asignacion	1 actividad conserva 0..* asignaciones. Cada asignación corresponde a 1 actividad.
Asignacion–Trabajador	1..* trabajadores por asignación, exactamente la cantidad requerida. 0..* asignaciones por trabajador.
Trabajador–Disponibilidad	1 trabajador compone 0..* intervalos. Cada intervalo pertenece a 1 trabajador.
Trabajador–Usuario	0..1 cuenta por trabajador y 0..1 trabajador por cuenta.
Actividad–ReporteAvance / Incidencia	1 actividad recibe 0..* reportes y 0..* incidencias.
Usuario–ReporteAvance	1 autor y 0..1 revisor por reporte. 0..* reportes por usuario.
Usuario–Incidencia / EventoAuditoria	1 autor por incidencia o evento. 0..* registros por usuario.
PlanSemanal–Actividad / Asignacion	Relaciones 0..* a 0..*. Las versiones publicadas conservan el compromiso original.

PlanificadorService coordinará sugerirCandidatos(), validarAsignacion() y confirmarAsignacion(); GrafoDependencias, detectarCiclo() y ordenar(); AgendaTrabajador, buscarConflicto(). Son servicios o estructuras auxiliares, no nuevas entidades del negocio.

El diagrama editable se encuentra en docs/uml/dizeo.drawio. El equipo deberá abrirlo, revisar ambas vistas y registrar fecha, participantes y cambios en docs/uml/REVISION.md. La revisión grupal permanece pendiente.

4. ALTERNATIVAS DE ESTRUCTURAS Y ALGORITMOS

Se comparan dos alternativas para priorizar tareas, controlar dependencias y detectar conflictos de agenda. Ambas aplicarán las mismas reglas de negocio. A servirá como referencia de corrección y B será la estructura principal seleccionada. Las cotas describen operaciones en memoria y no incluyen latencia de red, consultas SQL ni esperas de bloqueo.

Notación: V = actividades; E = dependencias; W = trabajadores; A = reservas activas totales; k = intervalos de un trabajador; q = tareas elegibles; c = candidatos por especialidad; r = reservas devueltas; S = relaciones trabajador–especialidad. Las agendas incluyen asignaciones y bloqueos aprobados. Las jornadas se validan por separado.

Alternativa	Estructuras y algoritmos	Ventajas y limitaciones
A Listas y matriz	ArrayList para entidades y agenda; matriz booleana V × V para precedencias; búsqueda lineal; ordenamiento por comparador; DFS recorriendo filas de la matriz.	Implementación directa y fácil de inspeccionar. Recorre la agenda y reserva memoria cuadrática para dependencias.
B Índices y grafo disperso	HashMap por ID; índice por especialidad; TreeMap por trabajador; PriorityQueue; grafo de listas de adyacencia; Kahn y consultas de vecinos.	Favorece consultas repetidas y dependencias dispersas. Exige mantener índices y agendas coherentes. No garantiza un óptimo global.

4.1. Agenda e identificación

Operación	A Tiempo / espacio adicional	B Tiempo / espacio adicional
Buscar trabajador por ID	O(W) / O(1)	O(1) esperado; O(W) conservador en peor caso / O(1).

Operación	A Tiempo / espacio adicional	B Tiempo / espacio adicional
Validar cruce de un trabajador	O(k) / O(1)	O(log k) / O(1), con intervalos no solapados.
Insertar reserva validada	O(1) amortizado / O(1)*	O(log k) / O(1)*.
Listar r reservas de un rango	O(k + r) / O(r)	O(log k + r) / O(r).
Filtrar candidatos compatibles y libres	O(W + A) / O(c), recorriendo agendas.	O(c log k máximo) esperado / O(c), con índice por especialidad.

* Espacio incremental lógico por reserva. Una expansión interna de ArrayList puede requerir O(k) temporal. El almacenamiento de agendas es O(W + A). Fuentes: Oracle (s. f.-a, s. f.-c). Los costos compuestos se derivan del diseño.

4.2. Prioridades y dependencias

Operación	Alternativa A	Alternativa B
Construir dependencias	$O(V^2 + E)$ tiempo y $O(V^2)$ memoria, incluida inicialización.	O(V + E) tiempo y memoria.
Detectar ciclo u obtener orden topológico	DFS con matriz: $O(V^2)$ tiempo y O(V) auxiliar además de la matriz.	Kahn: O(V + E) tiempo y O(V) auxiliar además del grafo.
Elegir y retirar próxima tarea elegible	Recorrido para mínimo y borrado: O(q) tiempo y O(1) auxiliar.	poll() en PriorityQueue: O(log q) tiempo y O(1) auxiliar. Cola: O(q).
Mostrar todas las tareas en orden	Ordenar copia: O(q log q) tiempo y O(q) auxiliar.	Copiar y extraer: O(q log q) tiempo y O(q) auxiliar. Iterar el heap no ordena.
Cambiar prioridad arbitraria	Encontrar y actualizar: O(q). Se vuelve a ordenar al consultar.	remove(objeto) + offer: O(q). Retirar antes de cambiar la clave.

Operación	Alternativa A	Alternativa B
Memoria total relevante	$O(W + A + S + V^2)$, sin reportes ni historial.	$O(W + A + S + V + E)$, sin reportes ni historial.

4.3. Supuestos de las cotas

TreeMap ordenará reservas por inicio. Si las reservas confirmadas no se solapan, bastará revisar el vecino anterior y el siguiente, además de los traslados desde la obra anterior y hacia la siguiente. Una agenda importada con cruces deberá corregirse antes de utilizar esa validación. Los bloqueos aprobados usarán el mismo índice y las jornadas se validarán por separado.

Dos intervalos [a,b) y [c,d) se cruzan cuando a < d y c < b. Si b = c, solo serán compatibles cuando el traslado requerido sea cero. La consulta por rango incorporará el intervalo anterior al inicio del rango si continúa dentro de él. Cada cambio de dependencias se validará mediante un recorrido completo O(V + E).

Kahn utilizará ArrayDeque para detectar ciclos. La cola de prioridad contendrá tareas elegibles para la operación elegida: planificar exige una secuencia prevista compatible e iniciar ejecución exige predecesoras terminadas. Kahn con selección mediante heap costaría O(E + V log V), distinto del recorrido FIFO O(V + E). Fuente del heap: Oracle (s. f.-b).

5. SELECCIÓN Y FUNDAMENTACIÓN

Se selecciona la alternativa B porque el problema requiere consultar repetidamente disponibilidad, precedencias y prioridades sobre datos que cambian. La alternativa A conservará valor como referencia de corrección en conjuntos pequeños. La elección se fundamenta en complejidad y adecuación al problema; todavía no existen mediciones del sistema implementado.

Criterio	Fundamentación de B
Adecuación	Representa identidad, tiempo y precedencia por separado. Conserva la decisión del supervisor y explica conflictos.
Tiempo	Validar un intervalo pasa de O(k) a O(log k); recorrer dependencias dispersas pasa de $O(V^2)$ a O(V + E).
Memoria	Con V = 10 000 y E = 30 000, A representa 100 millones de celdas lógicas y B 10 000 vértices y 30 000 aristas. Son conteos teóricos, no bytes medidos.
Escalabilidad	Favorece consultas frecuentes. Si E se aproxima a V^2, disminuye la ventaja del grafo disperso. Listar todo el orden sigue costando O(q log q).
Mantenibilidad	Encapsular índices y mantener PostgreSQL como fuente persistente. Actualizar índices después del commit.
Dificultad	Usar colecciones estándar de Java. Implementar comparadores, Kahn y validación temporal, sin construir un árbol rojo-negro desde cero.

5.1. Política de decisión

El supervisor definirá el intervalo y solicitará sugerencias. El sistema filtrará personas compatibles y libres, las ordenará por menor carga semanal y desempatará

por código. Ordenar c candidatos agrega O(c log c), además de calcular sus cargas. La heurística no garantiza minimizar retrasos ni equilibrar globalmente todas las obras. Si no se completa el personal requerido, la asignación no se confirmará.

5.2. Confirmación consistente

Un servicio Java central atenderá Swing y persistirá en PostgreSQL. Al confirmar, bloqueará las filas de trabajadores en orden de código, releerá las agendas, revalidará condiciones y guardará asignación, historial y versión dentro de una transacción. La cancelación y reprogramación utilizarán el mismo protocolo. Los índices se actualizarán después del commit. Esta decisión de diseño deberá verificarse en Progreso 2 (PostgreSQL Global Development Group, s. f.).

6. PLAN PRELIMINAR DE IMPLEMENTACIÓN

El trabajo se distribuye entre Daniel Valarezo, Esteban Figueroa y Julián Jiménez. Las responsabilidades incluyen los cinco módulos y los componentes transversales. La integración final, la revisión de documentos y las decisiones de diseño se realizarán entre los tres integrantes.

Integrante	Responsabilidades del proyecto	Evidencia prevista
Daniel Valarezo	M1 Obras y M5 Seguimiento. Formulación del problema, referencias y coordinación del informe.	Filtros y orden de obras, avance ponderado, cierre validado y documentación.
Esteban Figueroa	M2 Trabajadores y M4 Planificación. Disponibilidad, agendas, asignaciones e integración con persistencia.	Compatibilidad, calendario, sugerencias, cruces y confirmación consistente.
Julián Jiménez	M3 Actividades. Prioridades, grafo, UML y comparación de alternativas. Apoyo a integración y verificación.	Comparador, detección de ciclos, diseño orientado a objetos y fundamentación técnica.

6.1. Hitos relativos hacia Progreso 2

Hito	Trabajo y criterio de salida
Semana 1	Validar problema y alcance, revisar UML y contratos. Conservar observaciones docentes como requisitos de mejora.
Semana 2	Implementar dominio y estructuras de A y B; contrastar resultados en datos pequeños, ciclos, empates y límites horarios.
Semana 3	Integrar servicio, PostgreSQL y Swing. Verificar transacciones, permisos, auditoría y actualización de agendas.

Hito	Trabajo y criterio de salida
Semana 4	Pruebas integradas, rendimiento, concurrencia, documentación y ensayo entre los tres. Ajustar fechas al calendario del curso.

6.2. Estructura inicial del repositorio

Ubicación	Contenido
README.md	Descripción, integrantes, módulos, alternativa seleccionada y estructura.
docs/informe	Informe técnico en Word y PDF.
docs/presentacion	Presentación técnica con notas y reparto entre tres expositores.
docs/uml	Diagrama editable y registro de revisión grupal.
docs/plan	Observaciones docentes y mejoras para Progreso 2.

El repositorio Git inicial contiene documentación de análisis y diseño. No incorpora implementación del sistema. Se conserva un historial de documentación y el README explica la publicación en la cuenta de GitHub que utilice el grupo.

6.3. Riesgos técnicos y mitigación

Riesgo	Mitigación	Comprobación prevista
Doble reserva simultánea	Transacción central, bloqueo de trabajadores y relectura de agendas.	En 50 pares incompatibles, aceptar como máximo una confirmación por par.
Índices desactualizados	Actualizar después del commit y reconstruir al recuperar sesión.	Comparar agenda reconstruida con reservas vigentes.
Ciclos o secuencia incorrecta	Kahn antes de aceptar dependencias y validación de precedencias.	Rechazar A–B–C–A y el inicio con predecesoras sin terminar.

Riesgo	Mitigación	Comprobación prevista
Traslados omitidos	Comprobar ambos vecinos y exigir tiempos registrados.	A 08:00–12:00; B a las 12:00 se rechaza y a las 12:30 puede admitirse.
Prioridad obsoleta en heap	Retirar y reinsertar al cambiar la clave.	La actualización modifica el próximo elemento según prioridad, fecha y código.
Pérdida de observaciones	Registrar texto, responsable y evidencia de corrección.	Cada observación tendrá estado y cierre revisado para Progreso 2.

El entorno de aceptación heredado propone 2 núcleos y 4 GB, 100 obras, 200 trabajadores, 10 000 actividades y 20 usuarios simultáneos. Se prevén 100 solicitudes por operación, consultas con p95 ≤ 2 s y conflictos con p95 ≤ 3 s. Estos son objetivos de prueba, no resultados obtenidos. También deberán verificarse roles, credenciales, respaldo, auditoría y zona horaria.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Herramienta	Uso realizado	Resultado incorporado	Verificación
ChatGPT	Apoyo en análisis, redacción y diseño preliminar.	Borradores del informe, UML y presentación.	Contraste con fuentes y revisión grupal pendiente.

El equipo revisará, adaptará y comprenderá el contenido incorporado y asumirá la responsabilidad académica y técnica de la entrega.

7. PRESENTACIÓN TÉCNICA

Duración prevista: 9 minutos y 30 segundos, más preguntas. Los tres integrantes participarán aproximadamente 3 minutos y 10 segundos cada uno.

Diapositivas	Contenido	Expositor	Tiempo
1–2	Proyecto, problema y evidencia	Daniel	1 min 35 s
3–4	Variables, restricciones y módulos	Esteban	1 min 40 s
5–6	UML y alternativas	Julián	2 min
7	Comparación de complejidad	Julián	1 min 10 s
8	Selección y fundamento	Daniel	55 s
9	Caso de asignación	Esteban	1 min
10	Plan de implementación	Daniel y Esteban	40 s y 30 s

REFERENCIAS

Durdyev, S., & Hosseini, M. R. (2020). Causes of delays on construction projects: A comprehensive list. *International Journal of Managing Projects in Business, 13*(1), 20–46. https://doi.org/10.1108/IJMPB-09-2018-0178

Gómez Sánchez, M., Lalla-Ruiz, E., Fernández Gil, A., Castro, C., & Voß, S. (2023). Resource-constrained multi-project scheduling problem: A survey. *European Journal of Operational Research, 309*(3), 958–976. https://doi.org/10.1016/j.ejor.2022.09.033

Hendrickson, C. (2008). *Project management for construction: Fundamental concepts for owners, engineers, architects and builders* (versión 2.2, cap. 11). Carnegie Mellon University. https://www.cmu.edu/cee/projects/PMbook/11_Advanced_Scheduling_Techniques.html

Oracle. (s. f.-a). *HashMap (Java SE 21 & JDK 21)*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html

Oracle. (s. f.-b). *PriorityQueue (Java SE 21 & JDK 21)*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html

Oracle. (s. f.-c). *TreeMap (Java SE 21 & JDK 21)*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html

PostgreSQL Global Development Group. (s. f.). *Explicit locking*. PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/explicit-locking.html

Valarezo, D., Figueroa, E., & Jiménez, J. (2026). *Levantamiento de requerimientos para Dizeo: Sistema de planificación y asignación de trabajadores y actividades de obra* [Trabajo académico no publicado]. Universidad de las Américas.